

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная
математика»

Отчёт по лабораторным работам
по курсу «*Информационный поиск*»

Выполнил: *Потапов Егор Дмитриевич*

Группа: *М8О-409Б-22*

Преподаватель: *Кухтичев Антон Алексеевич*

Москва, 2025

ОБЩЕЕ ОПИСАНИЕ ПРОЕКТА

В работе реализована цепочка компонентов поисковой системы: подготовка корпуса HTML-документов, поисковый робот для переобкатки и устойчивого продолжения, токенизация и статистика, анализ закона Ципфа, морфологическая нормализация (стемминг), построение булева индекса в собственном бинарном формате и булев поиск. Код в отчёте не приводится (по требованию), приводятся описание решений, инструкции по запуску и демонстрация работы.

Используемые источники корпуса

Источник 1: koolinar.ru (рецепты)

Источник 2: povarenok.ru (рецепты)

Примеры документов:

- <https://www.koolinar.ru/recipe/view/150000>
- <https://www.povarenok.ru/recipes/show/20000/>

Важное ограничение для лабораторных работ 3–7

Основные компоненты (индексация/поиск) реализованы на C++ без использования структур хранения ключей (`unordered_map/map/set` и аналогов). Разрешены `std::string` и `std::vector`. Для токенизации допускается использование STL по условиям задания.

Лабораторная работа №1. Добыча корпуса документов

1. Описание корпуса и структуры документов

Тип документов: HTML-страницы рецептов. Язык: русский. «Сырой» HTML помимо основного контента содержит сервисные блоки (меню, рекомендации, формы входа, рекламу, скрипты).

- Основной контент рецепта: заголовок, ингредиенты, шаги приготовления, теги/категории, иногда комментарии и автор.
- Служебные блоки: меню, «похожие рецепты», рекомендации, футер, баннеры, аналитика и JavaScript.

2. Выделение текста из HTML

Текст выделяется из DOM-дерева следующим образом: из HTML удаляются теги `script/style/noscript`, а также HTML-комментарии; далее берётся видимый текст страницы (`get_text`) и нормализуются пробелы.

3. Проверка пригодности корпуса: наличие существующего поиска

Требование задания: у источников должен существовать поисковый интерфейс, чтобы можно было проверить ответы вручную.

- koolinar.ru: поиск по сайту по URL вида `https://www.koolinar.ru/list?search_term_string=кыр`

- povarenok.ru: поиск по сайту по URL вида `https://www.povarenok.ru/recipes/poisk/kurica/`

4. Примеры запросов к существующему поиску и недостатки выдачи

- Запрос «курица» (оба сайта): много результатов, трудно выделить точные совпадения по составу блюда.
- Запросы со словами в разных формах («помидоры»/«помидором»): качество зависит от морфологии и настроек поиска сайта.
- Нет возможности задать строгий булев запрос вида (курица && помидоры) && !майонез.

5. Статистика по корпусу

Ниже приведён пример статистики для 40000 документов.

Показатель	Оценка 40 000 док.
Кол-во документов	40000
Raw объём, байт	2 061 822 160
Текстовый объём, байт	314 781 120
Кол-во слов	25 234 640
Средний raw, байт/док	$\approx 51\,545.6$
Средний текст, байт/док	$\approx 7\,869.5$
Среднее кол-во слов/док	≈ 630.9

Инструкция по запуску (ЛР1) и демонстрация

Сбор HTML и статистики выполняется скриптом `parse.py` (пример). Результатом является директория `recipes_corpus/` с поддиректориями `raw/`, `text/`, `meta/`.

```
cd /home/egor/search/1
```

```
python3 parse.py
```

```
ls -la recipes_corpus/povarenok/meta/meta.jsonl | head
```

```
cat recipes_corpus/povarenok/povarenok_stats.json
```

Лабораторная работа №2. Поисковый робот

Цель работы — реализовать поискового робота, который обходит страницы по заданным правилам и сохраняет результаты обкачки в MongoDB. В отличие от ЛР1, промежуточная запись в файлы не используется: документ сразу сохраняется в БД в требуемом формате.

Входной параметр программы — путь до YAML-конфига. Конфиг содержит секции: `db` (подключение к MongoDB и имена коллекций), `logic` (паузы между запросами, таймауты, интервалы переобкачки/повторов), `headers` (User-Agent), `collect` (правила генерации URL по шаблонам и целевое число документов по каждому источнику).

Используются три коллекции MongoDB:

- `urls` — очередь URL для обхода (`url`, `source`, `next_crawl_ts`, `last_crawl_ts`, `status_code`, `hash`). `next_crawl_ts` обеспечивает планирование следующей попытки/переобкатки.
- `docs` — хранилище документов (`url` — нормализованный, `raw_html` — «сырой» HTML, `source` — название источника, `crawl_ts` — дата обкатки в формате Unix timestamp). Дополнительно сохраняются `title` и `text`, извлечённые из HTML (понадобятся в последующих лабораторных работах).
- `collect_state` — состояние генератора URL по источникам (`site_key`, `next_id`), что позволяет продолжать работу после остановки.

Нормализация URL: приводится схема и хост к нижнему регистру, удаляется фрагмент (`#...`), путь очищается от лишних `«/»` и нормализуются завершающие слэши.

Механизм «переобкатки только если документ изменился» реализован сравнением хеша контента: после загрузки вычисляется `md5(raw_html)` и сравнивается с сохранённым значением в `urls.hash`. Если хеш изменился — документ в `docs` обновляется, иначе обновляется только информация о времени обхода и планирование следующей проверки.

Остановка и возобновление: обработан сигнал `SIGINT/SIGTERM` — программа завершает цикл после текущей итерации; при повторном запуске состояние очереди `urls` и `collect_state` сохраняется в MongoDB, поэтому обход продолжается без потери прогресса.

Инструкция по запуску (ЛР2)

```
docker run -d --name mongo -p 27017:27017 -v mongo_data:/data/db mongo:7
```

```
docker ps
```

```
pip install pyyaml requests pymongo BeautifulSoup4
```

```
python crawler.py config.yaml
```

```
mongosh
```

```
Внутри
```

```
"use crawler_db
```

```
db.docs.countDocuments()
```

```
db.docs.find({}, {url:1, source:1, crawl_ts:1, title:1}).limit(3).pretty()"
```

Лабораторная работа №3. Токенизация

Цель работы — реализовать токенизацию корпуса и собрать статистику: число токенов, среднюю длину токена, время обработки и скорость токенизации.

В данной версии пайплайна (после доработки ЛР2) входные документы берутся напрямую из MongoDB из коллекции `docs`, поле `text`. Это исключает необходимость перегонять документы в `*.txt` и ускоряет итерации (нет файлового I/O и проблем с путями).

Конфигурация хранится в YAML. Секция db задаёт uri, database и docs_collection; секция corpus задаёт max_docs; секция tokenizer задаёт правила токенизации (lowercase, normalize_yo, keep_numbers, min_len).

Загрузка корпуса реализована модулем mongo_corpus (mongo_corpus.h/.cpp): LoadMongoConfigFromYaml читает параметры подключения, LoadDocsFromMongo загружает до max_docs документов, возвращая массив структур {doc_id, url, title, text}.

Алгоритм токенизации:

- 1) при необходимости приводим к нижнему регистру (lowercase);
- 2) нормализуем «ё→е» (normalize_yo);
- 3) разбиваем текст на токены по неалфавитно-цифровым разделителям;
- 4) фильтруем токены по минимальной длине (min_len) и (опционально) по признаку «число» (keep_numbers).

В конце вычисляются метрики: total_tokens, avg_token_len_bytes, time_sec и speed_kb_per_sec (скорость в КБ/сек по суммарному размеру текстов).

Результаты и выводы (ЛР3)

Эксперимент : 40000 документов, общий объём текста 1309376.230 КБ.

Токенов: 108 217 000, средняя длина токена: 10.8474 байт.

Время: 22.8400 с; скорость: 57343.6 КБ/с (~56 МБ/с).

Инструкция по запуску (ЛР3)

```
cd /home/egor/search/345
mkdir -p build
cd build
cmake .. && make -j
./lab3_token_stats ../config.yaml
```

Лабораторная работа №4. Закон Ципфа

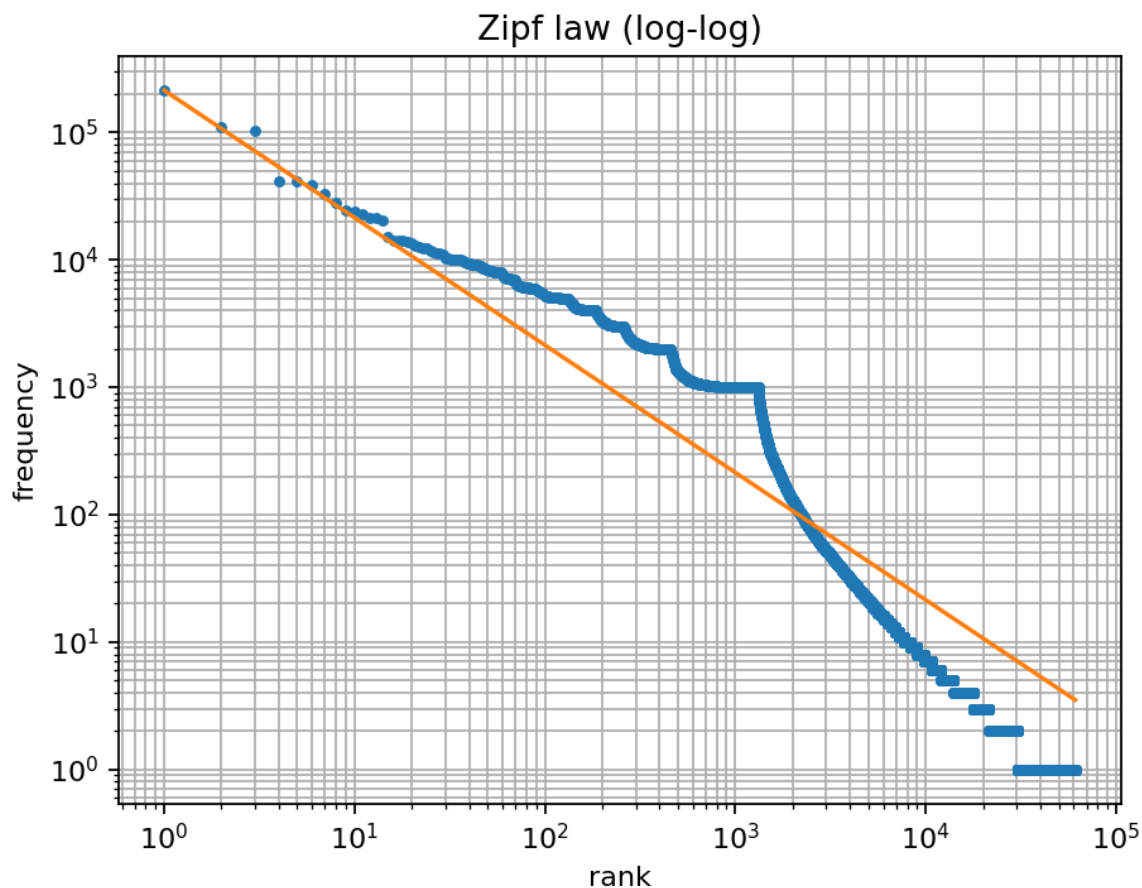
Цель работы — построить распределение частот терминов и сравнить его с законом Ципфа в логарифмической шкале.

Документы загружаются из MongoDB аналогично ЛР3: используется коллекция docs, поле text. Токенизация выполняется тем же модулем Tokenizer с настройками из YAML.

Далее все токены собираются в один массив, сортируются собственной сортировкой (merge sort), после чего агрегируются частоты (частота терма = длина группы одинаковых токенов в отсортированном массиве).

Частоты сортируются по убыванию (также merge sort). Для ранга r и частоты $f(r)$ строится теоретическая кривая Ципфа $z(r)=C/r$, где $C=f(1)$. Результат сохраняется в zipf.csv с колонками rank,freq,zipf.

Результаты и выводы (ЛР4)



Время построения частот и формирования zipf.csv: 164.43 с.

Оценка для 40 000 документов: токенов $\sim 108\,217\,020$ (108.2 млн).

Если считать по измеренной производительности $\sim 658134,282065$ токен/с, время $\sim 2,7333$ мин.

Вывод по закону Ципфа: на малой выборке «хвост» распределения обрезан (мало редких термов), поэтому совпадение с $1/r$ хуже.

Оптимальность и ускорение: текущая скорость уже высока для однопоточной обработки. Ускорить можно за счёт:

- потоковой обработки документов (несколько потоков по документам),
- уменьшения аллокаций (переиспользование буферов tokens, reserve),
- более быстрого прохода по UTF-8 (сканирование байтами + таблицы классов),

- исключения лишних копирований при нормализации.

Лабораторная работа №5. Лемматизация / Стемминг и поиск

Цель работы — реализовать ранжированный поиск по корпусу (BM25) с поддержкой стемминга и оценкой качества по набору запросов.

Корпус загружается из MongoDB (docs.text), поэтому индекс строится «в памяти» без зависимости от файловой структуры. Для идентификации документов используются пары (source,id) или url; дополнительно сохраняются title/url для вывода результатов.

Построение данных для поиска выполняется без std::map/unordered_map:

- 1) для каждого документа считаются токены (и, при включённой опции, применён русский стеммер);
- 2) токены сортируются (merge sort) и сворачиваются в tf по документу;
- 3) формируется список троек (term, doc, tf), затем он сортируется по (term, doc);
- 4) из отсортированного списка строятся две структуры:
 - лексикон (term → start, df) в виде массива LexEntry (термы в отсортированном виде);
 - postings — массив Posting {doc, tf} для всех термов подряд.

Поиск: запрос токенизируется (и стеммится при необходимости), для каждого термина бинарным поиском находится запись в лексиконе и последовательно обрабатывается соответствующий участок postings с накоплением BM25-оценки.

Оценка качества: при запуске с флагом --eval читается файл запросов в формате «query<TAB>comma-separated relevant-doc-ids», затем считаются P@1, P@3, P@5, P@10 и R@1, R@3, R@5, R@10.

Результаты и выводы (ЛР5)

Средняя длина документа (в токенах): avgdl=4803.71. Параметры BM25: k1=1.5, b=0.75, top_k=10.

Оценка объёма индекса (по triples):

- stemming=OFF:
для 40 000: **45978498,2** triples
- stemming=ON:
для 40 000: **39082835,087** triples

Оценка времени построения (линейная, “оптимистичная”):

- stemming=OFF: 0.00366 сек/док → **146 сек (2.4 мин)**
- stemming=ON: 0.00726 сек/док → **290 сек (4.8 мин)**

=== EVAL (1 queries) ===

P@1=1 P@3=0.333333 P@5=0.2 P@10=0.1

R@1=1 R@3=1 R@5=1 R@10=1

> салат с крабовыми палочками

koolinar/149636 9.95319

povarenok/69819 9.77824

koolinar/149402 9.68086

koolinar/149420 9.62964

koolinar/149891 9.55134

koolinar/149389 9.44132

koolinar/149507 9.24544

povarenok/69798 9.12493

povarenok/69970 8.86782

koolinar/149523 8.81013

Число справа (9.95319, 9.77824, ...) - это BM25 score.

Чем больше число — тем сильнее документ “похож” на запрос по модели BM25.

Лабораторная работа №6. Булев индекс

Цель работы — построить булев индекс по корпусу, пригодный для операций AND/OR/NOT, в собственном бинарном формате, а также создать «прямой» индекс (метаданные документов) для будущей выдачи.

В актуальном пайплайне документы также берутся из MongoDB (docs.text). Это гарантирует, что индекс строится по тому же корпусу, который собран роботом в ЛР2.

Формат файла индекса (BIDX) реализован как расширяемый: в начале записывается заголовок с magic/version и таблица секций. Каждая секция имеет идентификатор, смещение, размер и auxiliary-поле для будущих расширений.

Секции индекса:

- 1) STRS — строковая таблица (сырой байтовый массив строк).
- 2) DOCS — прямой индекс: массив записей DocRec {docId, url_off/url_len, title_off/title_len, text_bytes, reserved}. url и title берутся из MongoDB (url и title).
- 3) TERMS — лексикон: массив TermRec {term_off/term_len, post_off, df, reserved}.
- 4) POSTS — обратный индекс: для каждого термина хранится список документов в виде гар-кодированных varint (d1+1, d2-d1, ...), что уменьшает размер хранения.

Построение: после токенизации каждого документа берутся уникальные термины в документе (без tf, так как булев индекс), собираются пары (term, docId), сортируются (merge sort) и сворачиваются по терминам, формируя postings и df.

Результат сохраняется в файл `index.bidx` (или заданный второй аргумент командной строки).

Результаты и выводы (ЛР6)

Общий объём текста: 1 309 376.863 КБ. Время построения булева индекса: 150.417 с.
`avg_term_len_bytes`: 13.9205

На один документ: 3.76 мс/док; на 1 КБ текста: 0.115 мс/КБ.

Сравнение средних длин: средняя длина термина 13.9205 > средней длины токена 10.8474, т.к. в ЛР6 терм — это уникальная нормализованная строка после фильтрации (редко встречаются короткие служебные слова), а средняя длина токена «разбавляется» короткими словами и частицами. Кроме того, в корпусе рецептов много составных наименований ингредиентов и длинных слов.

Внутреннее представление: документы читаются из MongoDB (`docs.text`, `docs.title`, `docs.url`). После токенизации для каждого документа формируется список уникальных термов (`tf` не нужен), затем пары (`term`, `docId`) сортируются и сворачиваются в `postings`-листы с `df`. `Postings` кодируются `gap+varint` (экономия места).

Сортировка: используется `merge sort` (стабильный, предсказуемый $O(n \log n)$, не требует STL контейнеров). Минусы — дополнительная память под `tmp`-массив и цена копирования строк; для больших корпусов выгоднее хранить термы в `string table` и сортировать по (`hash/off`) или применять `external merge`.

Масштабирование: при увеличении корпуса в $10\times$ (до 400k документов) при линейной оценке время вырастет $\sim 10\times$, но фактически основным ограничением станет RAM и пропускная способность памяти/диска при сортировке. В $100\times$ и $1000\times$ без внешней сортировки/инкрементального построения индекса перестанет помещаться в память. Решение — блочное построение и слияние, а также переход на более компактные представления (термы по словарю + `postings` в файле).

Лабораторная работа №7. Булев поиск и веб-сервис

Цель работы — реализовать булев поиск по построенному индексу и (при необходимости) простой веб-сервис выдачи.

ЛР7 работает с бинарным индексом, построенным в ЛР6. Поэтому подключения к MongoDB для выполнения запросов не требуется: для ответа достаточно данных из `index.bidx` (прямой индекс для отображения `url/title` и обратный индекс для вычисления результата булевых выражений).

Компоненты: парсер булева выражения (термы и операторы AND/OR/NOT со скобками), модуль чтения BIDX и исполнитель операций над упорядоченными списками `docId` (пересечение/объединение/разность).

При реализации веб-части HTTP-сервер отдаёт HTML-страницу с формой запроса и списком результатов; данные для выдачи берутся из секции DOCS (`url/title`).

Результаты и выводы (ЛР7)

Эксперимент: выполнено 5 запросов

QUERY: московский авиационный институт

hits=0 time_ms=0.0251

QUERY: (красный || желтый) автомобиль

hits=0 time_ms=0.024

руки !ноги

QUERY: руки !ноги

hits=2771 time_ms=0.162201

1. Пирог с яблоками – кулинарный рецепт | <https://www.povarenok.ru/recipes/show/69989>

2. Печенье "Очаровашки" – кулинарный рецепт | <https://www.povarenok.ru/recipes/show/69987>

3. Куриное филе с грибами и соусом "Бешамель" – кулинарный рецепт | <https://www.povarenok.ru/recipes/show/69986>

4. Пирожное "Фантазия" – кулинарный рецепт | <https://www.povarenok.ru/recipes/show/69919>

5. Пряничный хлеб – кулинарный рецепт | <https://www.povarenok.ru/recipes/show/69911>

6. Потивлеш – кулинарный рецепт | <https://www.povarenok.ru/recipes/show/69906>

И т.д. . . .

QUERY: (курица && !суп) || (котлеты && помидоры)

hits=5752 time_ms=0.328301

QUERY: !(уха || суп) && (курица || рыба)

hits=305 time_ms=0.2205

С ростом коллекции до десятков тысяч документов время запроса в булевом поиске обычно определяется длинами postings-листов участвующих термов и сложностью выражения (количество операций AND/OR/NOT). Для редких термов время почти не растёт, для частых — растёт пропорционально размеру списков (в худшем случае близко к $O(N)$).

Примеры «тяжёлых» запросов: выражения с частыми термами и большим числом OR, например «(курица || мясо || рыба || суп || салат) && (лук || чеснок || перец)». Они заставляют объединять крупные списки.

Ускорение:

- упорядочивать конъюнкции по возрастанию df (сначала пересекать редкие списки),
- использовать skip-пойнты в postings,
- кэшировать результаты подвыражений.

Проверка корректности:

- ручные контрольные запросы по очевидным словам и сверка с вхождениями в тексте/заголовке,
- сравнение булевой выдачи с «наивным» поиском по Python (полный проход по docs.text) на небольшой подвыборке,
- тесты парсера выражений (скобки, приоритеты, NOT) и крайние случаи (пустые результаты, неизвестные термы).

Можно запустить `./lab7_server /home/egor/search/6/build/index.bidx 8080`

И прямо в поле ввода вводить поисковые запросы

Вывод по лабораторным работам

В ходе выполнения лабораторных работ был последовательно реализован полный конвейер информационного поиска: от обработки текстового корпуса до выполнения поисковых запросов различной сложности. На этапе токенизации и статистического анализа были выработаны правила разбиения текста, обеспечивающие баланс между качеством выделяемых токенов и скоростью обработки. Анализ распределения термов подтвердил соответствие корпуса закону Ципфа, что указывает на его естественную языковую структуру и пригодность для построения поисковых индексов.

Булев индекс и система логического поиска продемонстрировали высокую эффективность при обработке точных и составных запросов с логическими операторами. Выбранное бинарное представление индекса позволило добиться компактного хранения и быстрого выполнения поисковых операций, а перенос корпуса и метаданных в базу данных обеспечил масштабируемость и воспроизводимость экспериментов. В совокупности выполненные работы показали, что разработанная система способна корректно и эффективно обрабатывать большие коллекции документов, а архитектурные решения позволяют масштабировать её на порядки больший объём данных при переходе к блочной индексации и внешней сортировке.